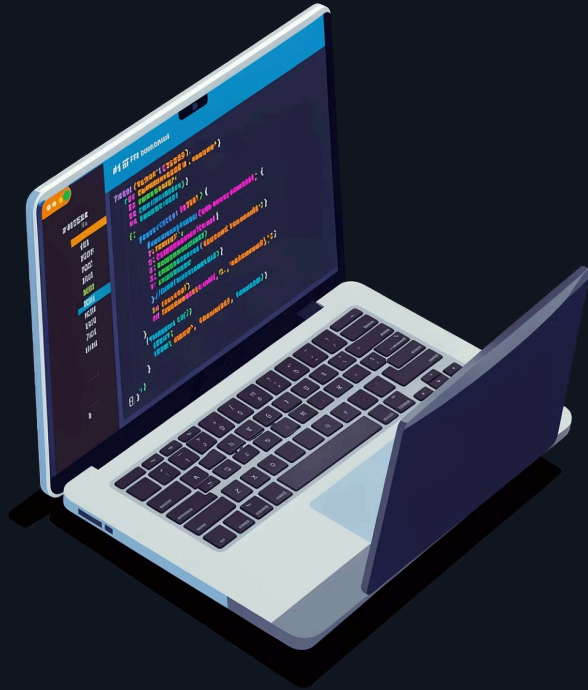




Car Rental Fleet Manager

Smart fleet operations for modern rental businesses

Presented by: Ridhi Chavan | 2620030032



Abstract

The **Car Rental Fleet Manager** is a console-based Java application designed to manage a small fleet of rental cars. It enables users to view available vehicles, search for specific cars, check availability status, rent and return cars, calculate rental charges, and organize fleet information through a menu-driven interface.

- ❏ **Key Focus:** Demonstrates fundamental Java programming and problem-solving concepts using **Java Foundations , Control Statements , Methods and Arrays**. Through practical application of variables, operators, control structures, arrays, and methods.

Key Advantages of the Car Rental Fleet Manager

The Car Rental Fleet Manager system offers a multitude of benefits, from enhancing operational efficiency to serving as a robust learning platform for Java development.



Optimized Operations

Streamlines vehicle allocation, maintenance scheduling, and overall fleet oversight, significantly reducing manual effort and administrative overhead.



Automated Billing

Automatically computes accurate rental charges based on duration and vehicle type, ensuring transparent pricing and eliminating manual calculation errors.



Scalable Design

Engineered with expandability in mind, allowing for seamless integration of new features, an increased fleet size, or additional functionalities as business demands evolve.



Core Java Demonstration

Effectively showcases proficiency in fundamental Java concepts such as data types, control flow, methods, and array manipulation within a functional system.



Easy Maintenance & Updates

The clear, organized structure facilitates quick identification of issues and allows for straightforward implementation of future enhancements or modifications.



Real-time Availability

Provides instant, up-to-date information on car availability, enabling quick booking decisions and preventing scheduling conflicts or overbooking.



User-Friendly Interface

A simple, console-based menu ensures the system is intuitive and easy to navigate for all users, minimizing the need for extensive training.



Practical Learning

Offers a tangible, real-world application for students to apply and solidify their understanding of core Java programming principles and problem-solving techniques.



Modular Architecture

Designed with a clear separation of concerns, making the codebase easier to understand, debug, and maintain, promoting reusability and structured development.



Enterprise Foundation

Provides a solid architectural base that can be extended and adapted into more complex, full-featured enterprise resource planning (ERP) solutions.

Problem Identification & Objectives

The Problem

Manual car rental management makes it difficult to track availability, pricing, customer selections, and rental duration efficiently.

IPO Model

Input → User menu choice

Process → Fleet operations

Output → Results & reports

Project Objectives

- **Maintain car information**
- **Track availability status**
- **Search vehicles by ID or name**
- **Manage rentals & returns**
- **Calculate costs, sort by price**

Module-Wise Concept Mapping

Foundations

- Computational thinking & algorithm design
- Variables, data types, operators
- Scanner, console I/O, formatted output
- Flowchart construction

Control Flow

- if-else, switch statements
- for, while, do-while loops
- break, continue, nested loops
- Decision-making structures

Methods & Arrays

- Methods, parameters, return values
- Method overloading
- Arrays & enhanced for loop
- Searching & sorting algorithms

System Design – Data Structures

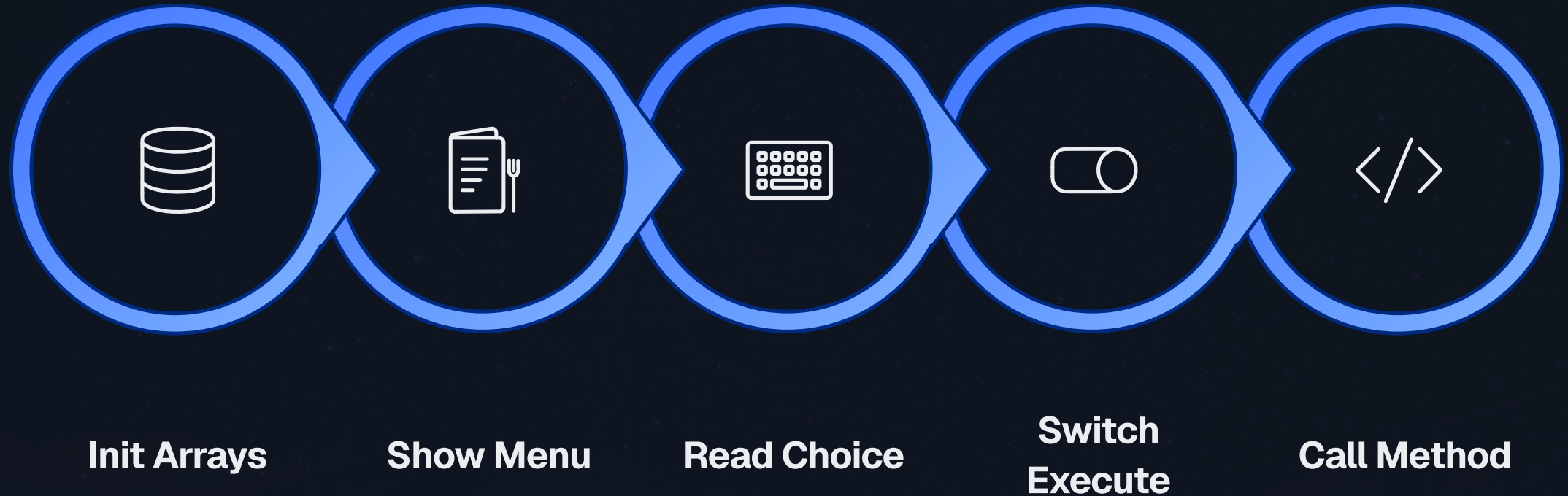


Parallel Arrays for Fleet Management

Array	Type	Purpose
carId[]	int	Unique IDs (101–105)
carName[]	String	Vehicle models
pricePerDay[]	int	Daily rental rate (₹)
available[]	boolean	Availability status
rentedDays[]	int	Rental duration tracker

All data is stored in **primitive arrays** — no Collections or Classes (Module 1–3 scope only).

Algorithm & Program Flow



The program uses a **do-while loop** to repeatedly show the menu until the user selects Exit, with a **switch statement** routing each choice to its corresponding method.

Implementation – Key Methods

1

displayCars()

Iterates arrays using a for loop; prints all fleet details with formatted output.

2

searchCar(int id) / searchCar(String name)

Method Overloading — linear search by ID or by name; returns index or -1.

3

rentCar() / returnCar()

Checks availability with nested if-else; updates boolean and rentedDays arrays.

4

calculateRent(price, days) / calculateRent(price, days, discount)

Method Overloading — standard cost or discounted cost calculation.

5

sortCarsByPrice()

Implements **Bubble Sort** across all parallel arrays simultaneously.

Complete Java Implementation

Module 1 Concepts Used
Variables, primitive data types (int, boolean, String), Scanner, arithmetic & relational operators, formatted console output

Module 2 Concepts Used
switch for menu routing, if-else for validation, for & do-while loops, nested conditionals, break to control flow

Module 3 Concepts Used
Static methods with parameters & return values, method overloading (searchCar, calculateRent), arrays, linear search, bubble sort

```
import java.util.Scanner;

public class CarRentalFleetManager {

    // MODULE 1: Primitive arrays — variables & data types
    static int[] carId = {101, 102, 103, 104, 105};
    static String[] carName = {"Swift","Baleno","Creta","City","Nexon"};
    static int[] pricePerDay = {1500, 1700, 2500, 2200, 1800};
    static boolean[] available = {true, true, false, true, true};
    static int[] rentedDays = {0, 0, 0, 0, 0};

    // MODULE 3: Method — display all fleet cars
    static void displayCars() {
        System.out.println("\n===== AVAILABLE CARS =====");
        for (int i = 0; i < carId.length; i++) { // MODULE 2: for loop
            System.out.println("ID: " + carId[i] + " | " + carName[i]
                + " | Rs." + pricePerDay[i] + "/day | "
                + (available[i] ? "Available" : "Rented")); // MODULE 2: ternary
        }
    }

    // MODULE 3: Method overloading — search by ID (linear search)
    static int searchCar(int id) {
        for (int i = 0; i < carId.length; i++) {
            if (carId[i] == id) return i; // MODULE 2: if
        }
        return -1;
    }

    // MODULE 3: Method overloading — search by name
    static int searchCar(String name) {
        for (int i = 0; i < carName.length; i++) {
            if (carName[i].equalsIgnoreCase(name)) return i;
        }
        return -1;
    }

    // MODULE 3: Method overloading — calculate rent (no discount)
    static int calculateRent(int price, int days) {
        return price * days; // MODULE 1: operator *
    }

    // MODULE 3: Method overloading — calculate rent (with discount)
    static int calculateRent(int price, int days, int discountPercent) {
        int total = price * days;
        int discount = (total * discountPercent) / 100;
        return total - discount;
    }

    // MODULE 3: Method — rent a car
    static void rentCar(Scanner sc) {
        System.out.print("Enter Car ID: ");
        int id = sc.nextInt(); // MODULE 1: Scanner I/O
        int index = searchCar(id);
        if (index == -1) { // MODULE 2: if-else
            System.out.println("Car not found!");
        }
        return;
    }
}
```

Program Output & Sample Execution

Let's walk through typical interactions with the Car Rental Fleet Manager, showcasing its core functionalities from main menu to fleet summary.

1

Main Menu & Initial Display

```
=== CAR RENTAL FLEET MANAGER ===
1.Display 2.Search 3.Rent 4.Return
5.Calc Rent 6.Sort 7.Summary 8.Exit
Choice: 1

===== AVAILABLE CARS =====
ID: 101 | Swift | Rs.1500/day | Available
ID: 102 | Baleno | Rs.1700/day | Available
ID: 103 | Creta | Rs.2500/day | Rented
ID: 104 | City | Rs.2200/day | Available
ID: 105 | Nexon | Rs.1800/day | Available
```

2

Searching for Cars

```
Choice: 2
Enter Car ID: 101
Found: Swift Rs.1500

Choice: 2
Enter Car ID: 999
Not found.
```

3

Renting a Car

```
Choice: 3
Enter Car ID: 101
Enter number of days: 3
Car rented! Total Cost: Rs.4500
```

4

Fleet Summary

```
Choice: 7

===== FLEET SUMMARY =====
Total: 5 | Available: 3 | Rented: 2 | Revenue: Rs.4500
```

This demonstrates how users can interact with the system to manage car rentals, query fleet status, and calculate costs efficiently.

Rental Cost Calculation Example



Scenario: Renting a Hyundai Creta

Parameter	Value
Car	Creta (ID: 103)
Rental Price	₹2500 / day
Duration	3 days
Total Cost	₹7,500

Method Call: `calculateRent(2500, 3)` → returns 7500

===== CAR RENTAL FLEET MANAGER =====

Choice: 3

Enter Car ID: 103

Enter number of days: 3

Car rented! Total Cost: Rs.7500

===== FLEET SUMMARY =====

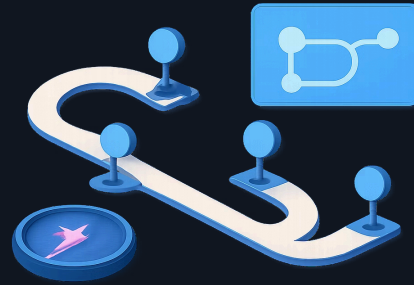
Total: 5 | Available: 3 | Rented: 2 | Revenue: Rs.12500

❏ Uses **arithmetic operator (*)**, **Scanner** for input, **if-else** for validation, and **calculateRent()** with parameters and return value — covering Modules 1, 2, and 3.

Conclusion & Key Takeaways

- Successfully integrates essential Java programming principles into a single practical application.
- Demonstrates how foundational concepts can work together to solve a real-world problem in a structured and efficient way.
- Applies key programming fundamentals, including variables, data types, operators, control statements, loops, arrays, methods, and input handling.
- Reinforces modular programming and logical problem-solving through reusable, method-based design.
- Highlights core programming constructs such as menu-driven user interaction, input validation, and iterative processing.
- Shows the practical value of Java for building console-based applications that are scalable, organized, and easy to maintain.
- Provides a strong foundation for future development into more advanced software solutions.





Future Work & Enhancements

Building upon the foundational concepts, several extensions can elevate the Car Rental Fleet Manager into a more robust and feature-rich application.



Graphical User Interface

Transition from console-based interaction to an intuitive GUI for enhanced user experience and accessibility.



Data Persistence

Implement file handling or database integration to store car and rental data persistently, beyond runtime.



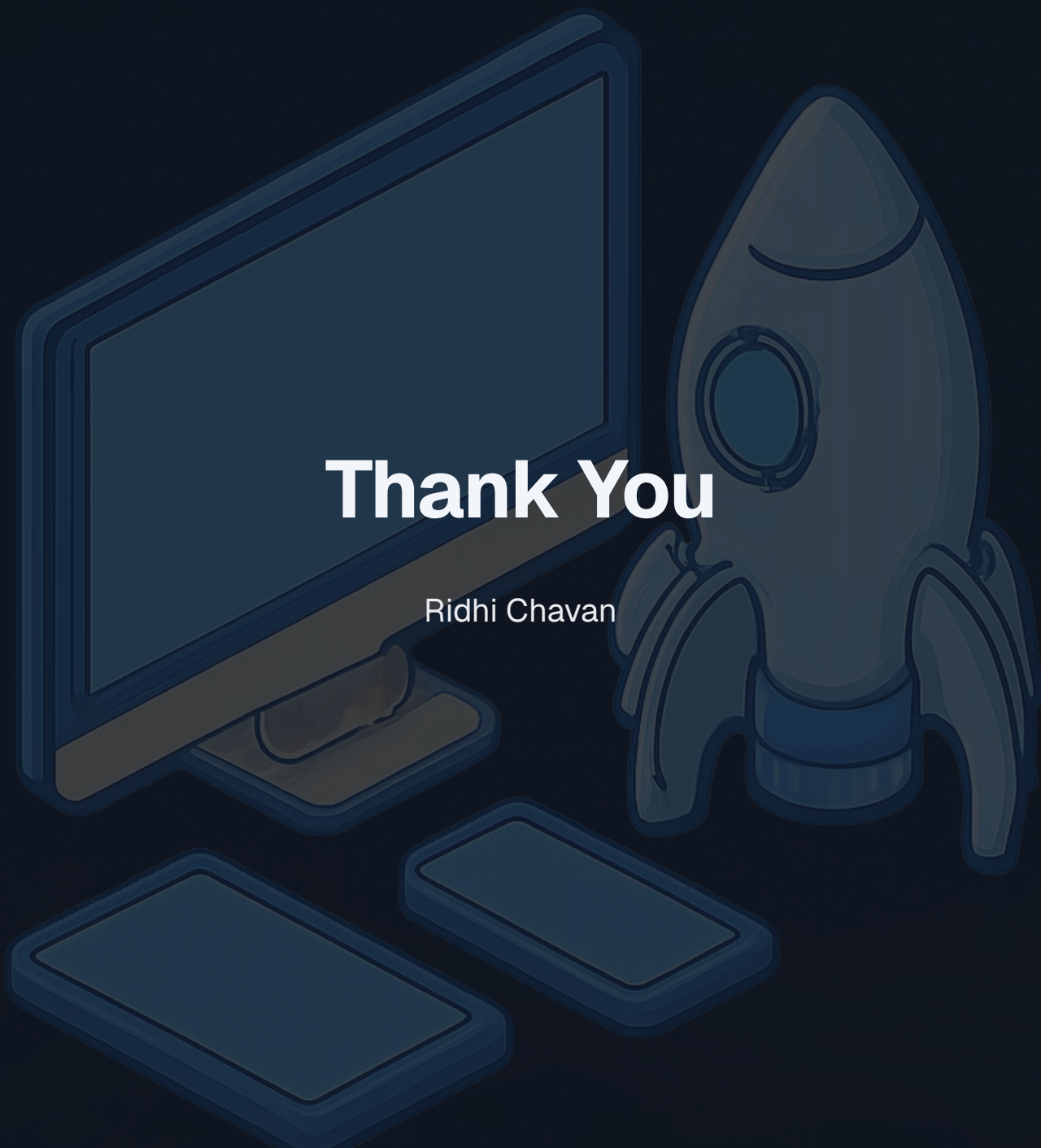
Robust Exception Handling

Integrate comprehensive exception handling to gracefully manage invalid inputs and potential system errors.



Advanced OOP Design

Utilize inheritance and polymorphism for different vehicle types (e.g., luxury, economy cars) and user roles.

An isometric illustration on a dark blue background. It features a computer monitor on the left, a rocket ship on the right, and two tablets at the bottom. The text 'Thank You' is centered over the monitor.

Thank You

Ridhi Chavan